Developer Portal Connectivity to Azure API Gateway

Azure-Gateway

Avudai Selvi S Last update: Jan 2022, Created: Jan 2022

)verview

ebMethods developer portal is simplified and lean architecture that is easier to run ıd requires less resources

It is headless, pluggable, and extensible in nature. There's an API for everything, customers will be able to use the Developer Portal or build their own UI for different type of asset.

The developer portal is event-enabled.Anything that happens in the product emits an event which other solutions can subscribe to.

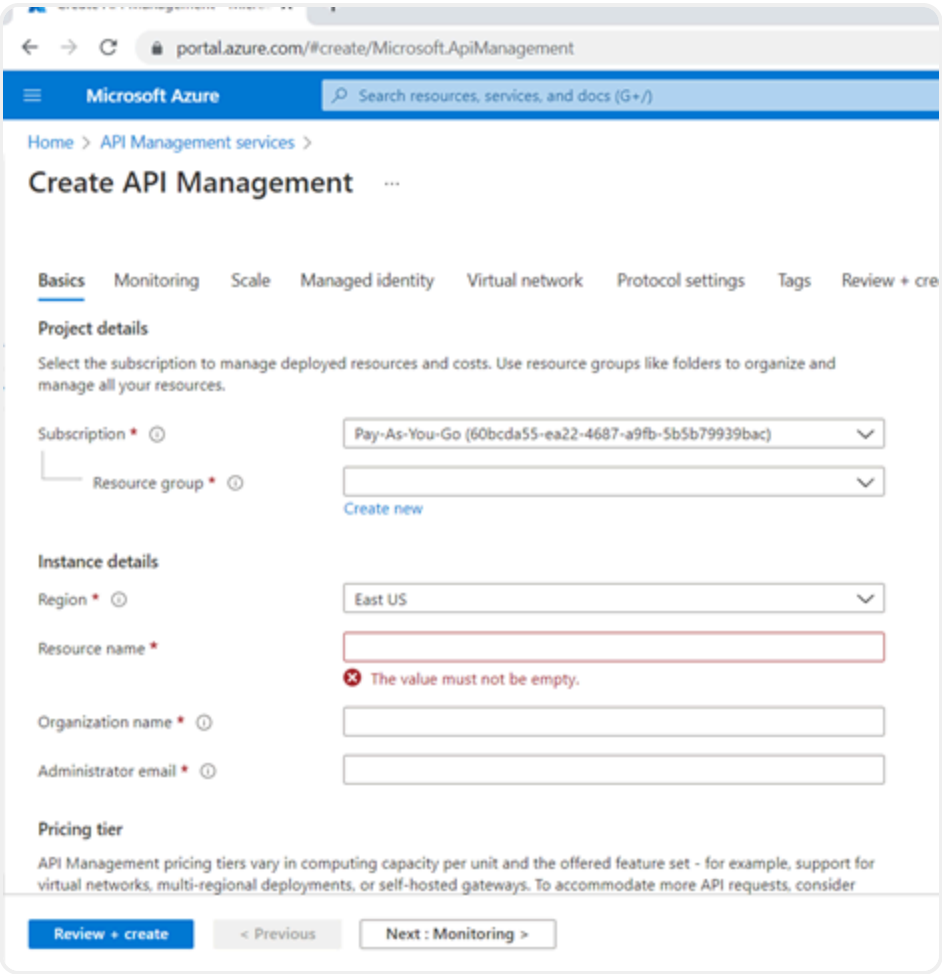
You can connect the Developer Portal with gateways from other vendors, or you can publish third-party APIs

et us consider the use case of integration with 3rd party gateways. APIs from different ateways can be exported and imported into the developer portal. Both external and ternal consumers can then consume the APIs. We can also have an optional tegration layer to automate the configurations required on the gateway side to stablish the connectivity with developer portal.

Skip to main content

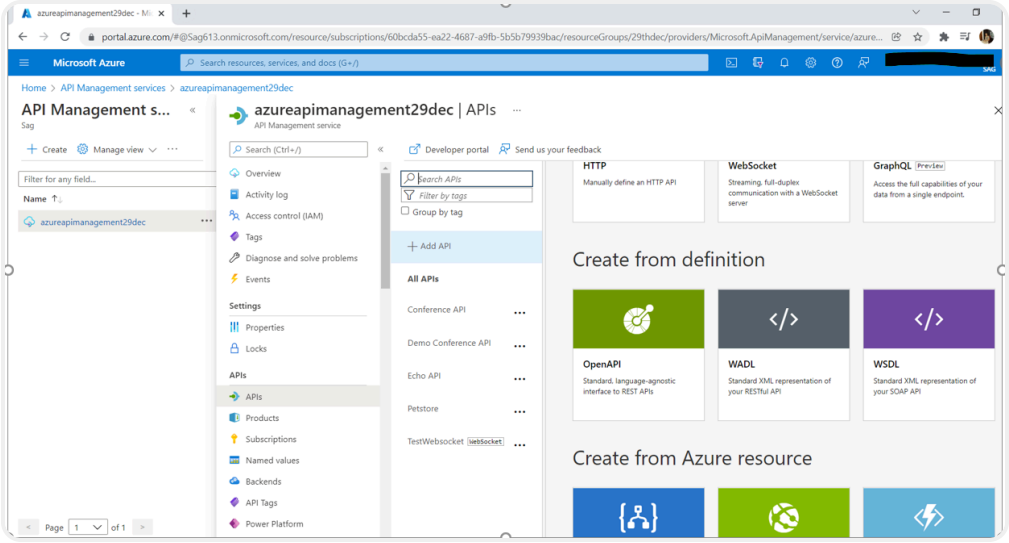
ızure Gateway Set Up

1. Create API Management Service in your azure account with proper resource group and name.

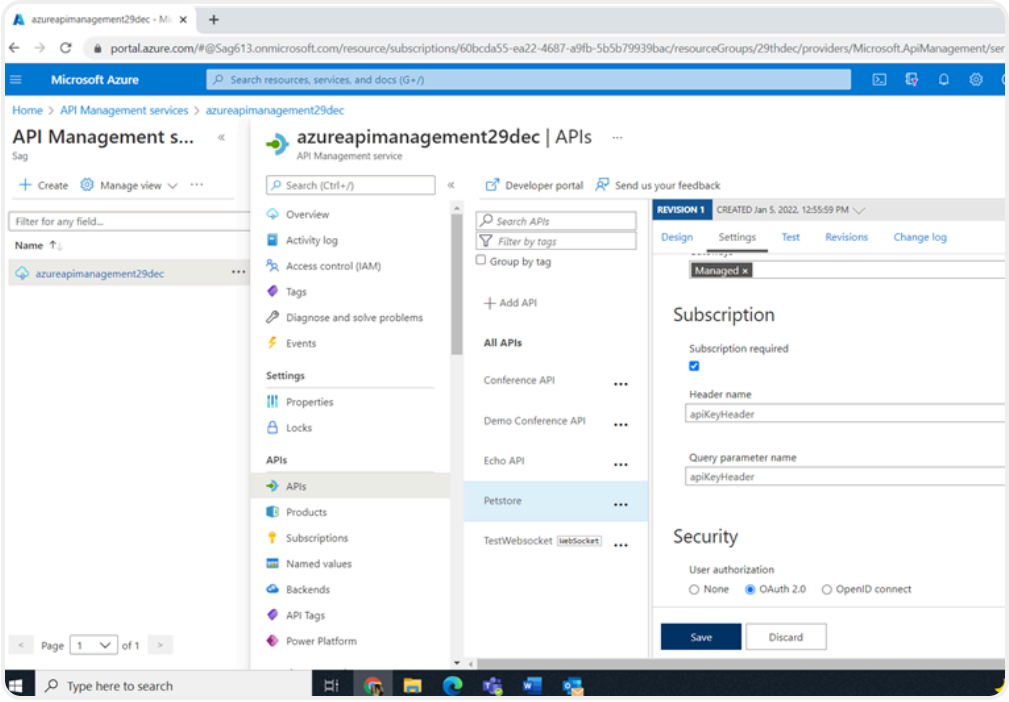


2. Once the API management service is create successfully. Create an API by clicking on the Add API.

Skip to main content

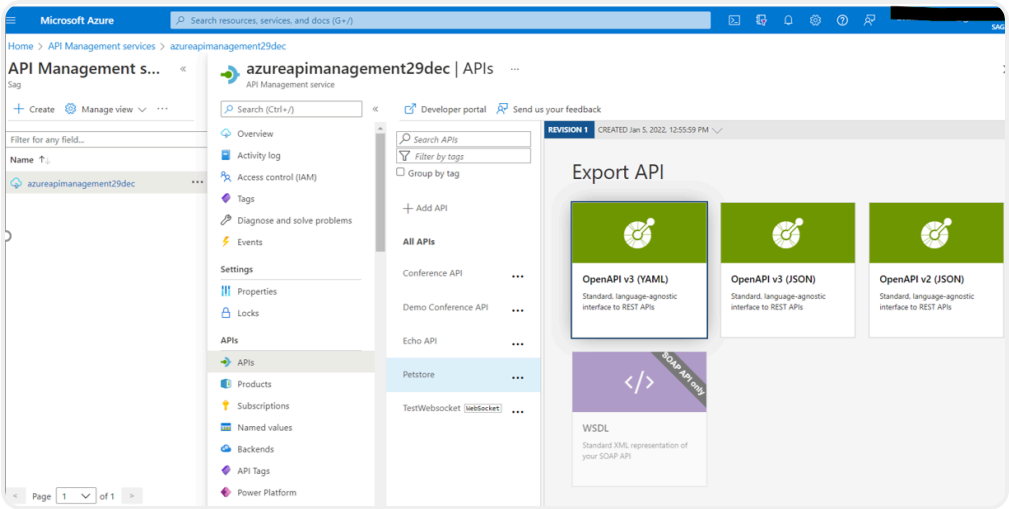


3. Once the API is created enable the subscription key and OAuth2.0 in the settings tab.



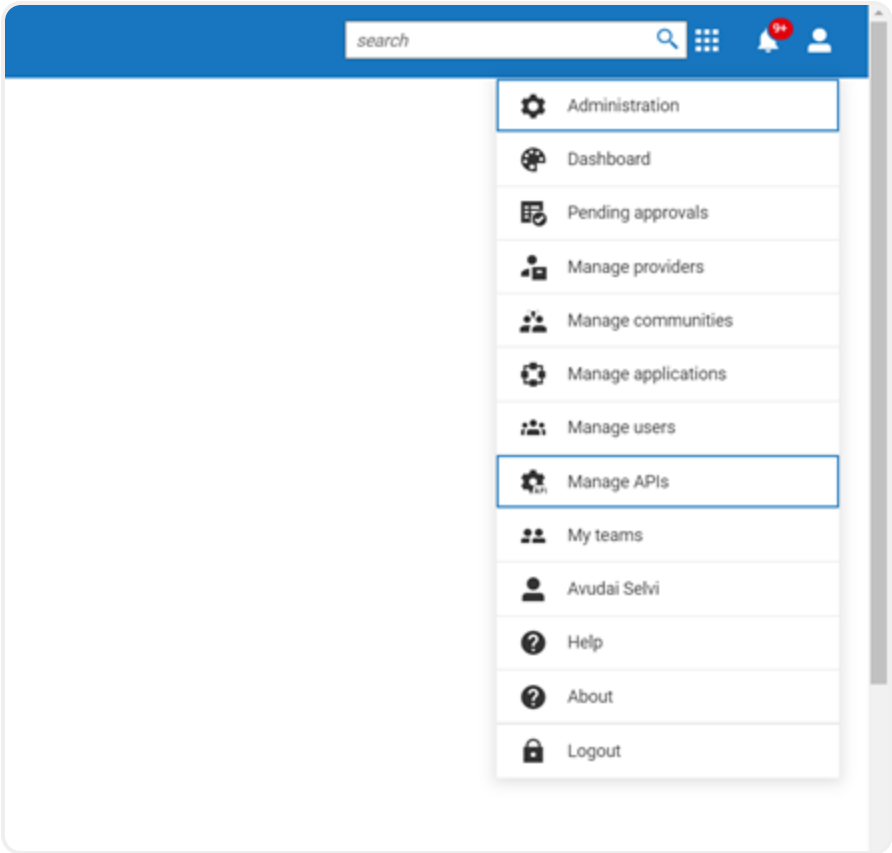
4. Export the API by clicking on the 3 dots near the API name and choose the format in which you want to export the API.

Skip to main content



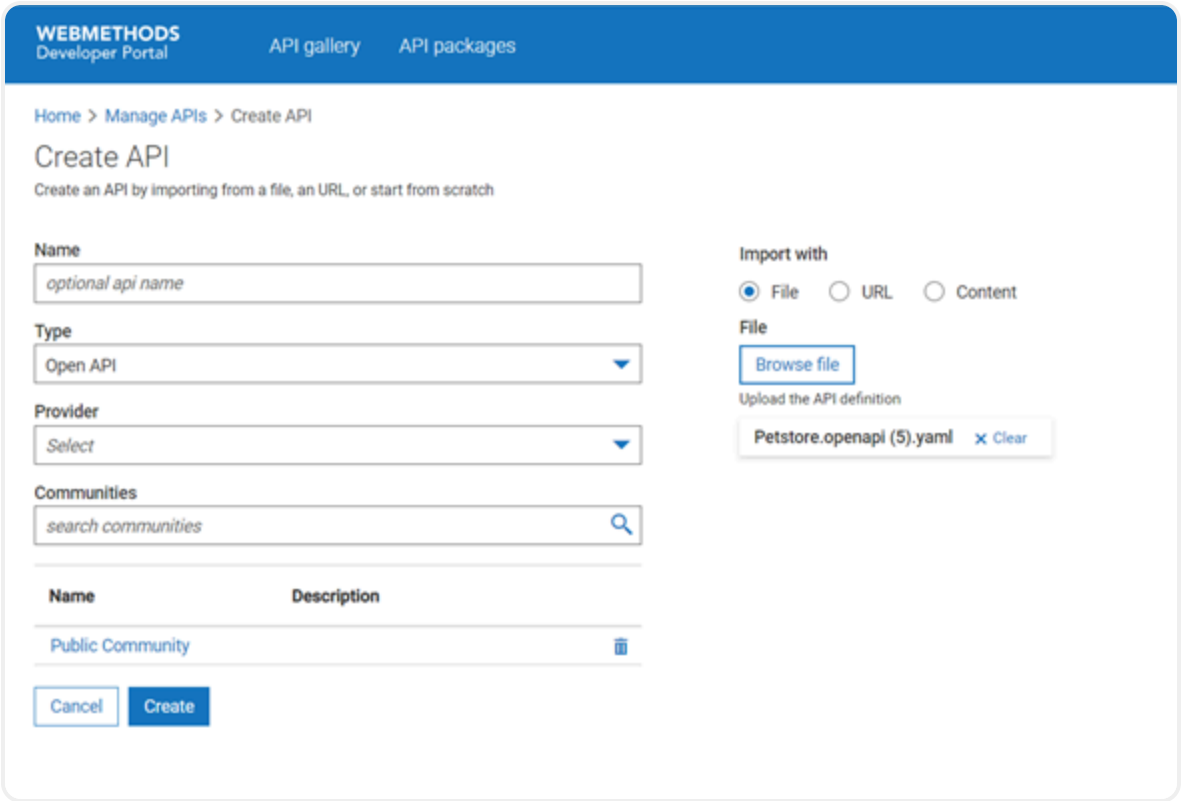
Developer Portal Set Up

1. Import the API exported from Azure in Developer portal by using the Manage APIs option.



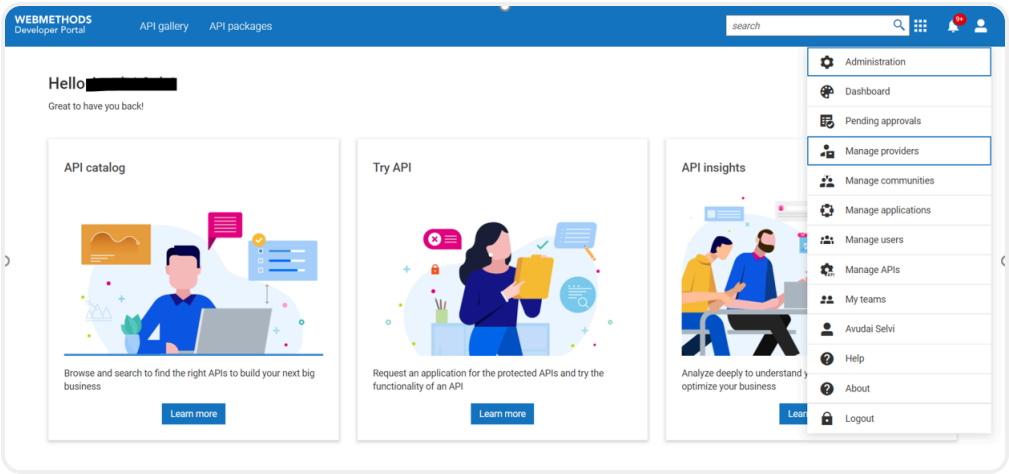
Skip to main content

2. Import the file to create the API in developer portal.



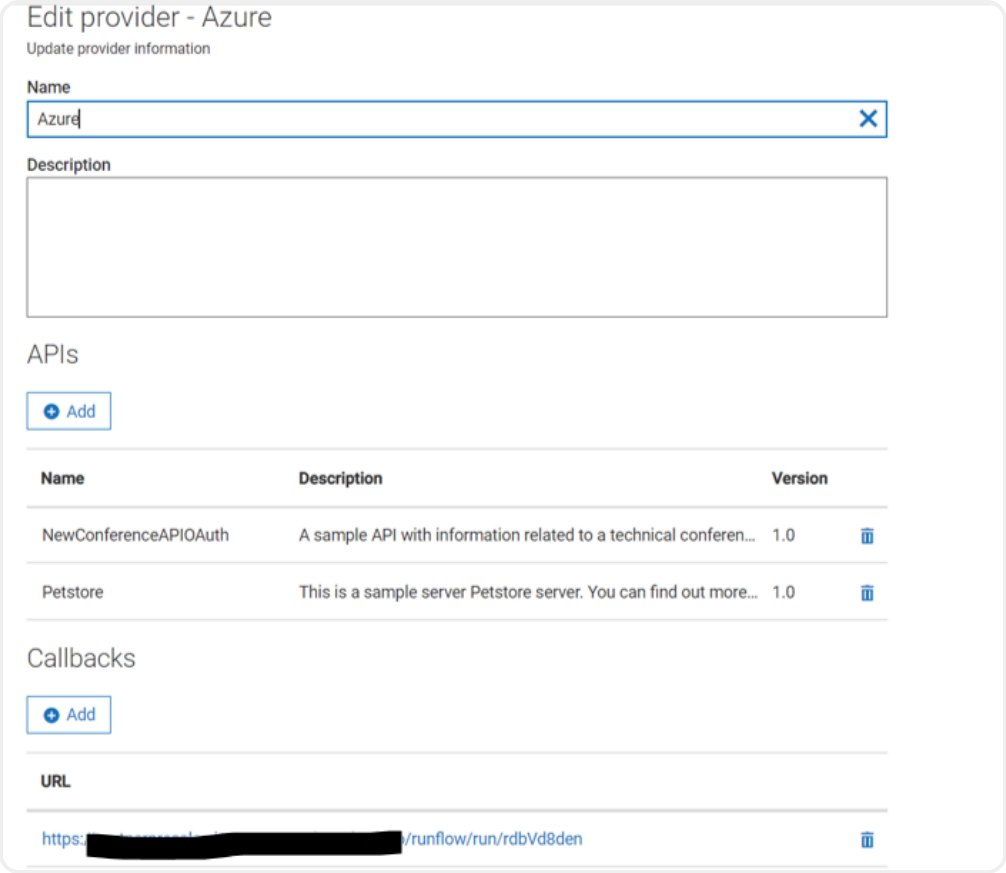
Once the API is created successfully the API should be tagged to a correct API provider.

3. 3rd Party Gateway providers should be configured as a provider in developer portal. Click on Manage providers.



4. Enter the provider details such as the provider name and select the callback url.

Skip to main content

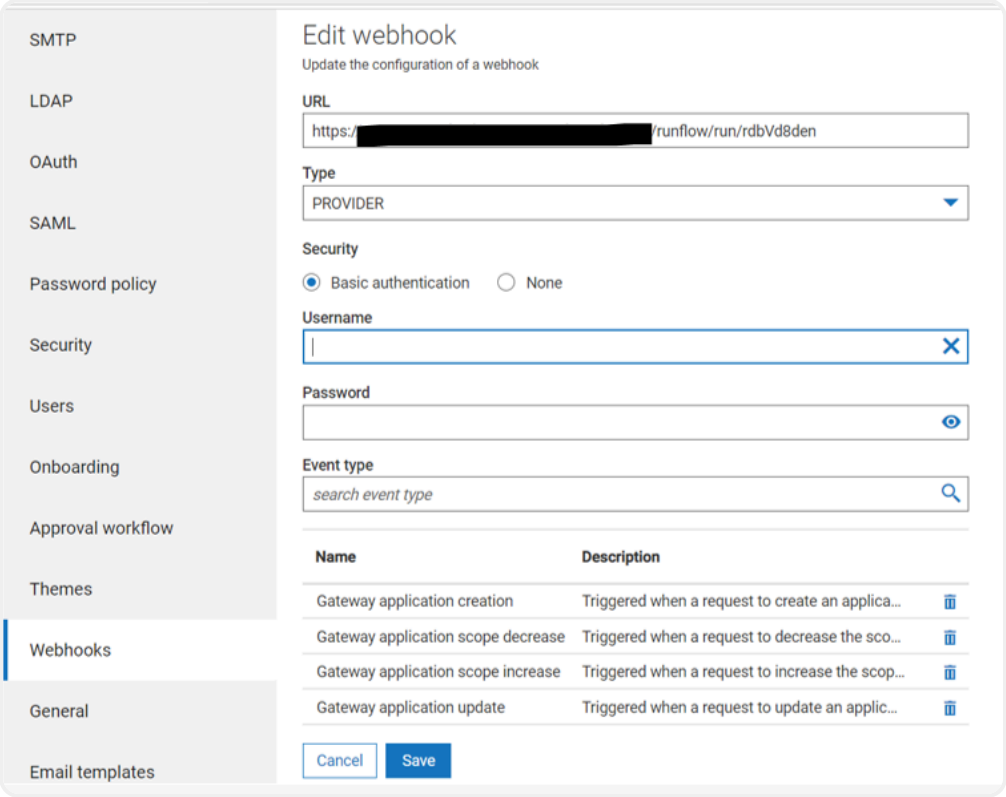


allback url will contain the end point detail to which the events will be published when onsumer take any action in developer portal.

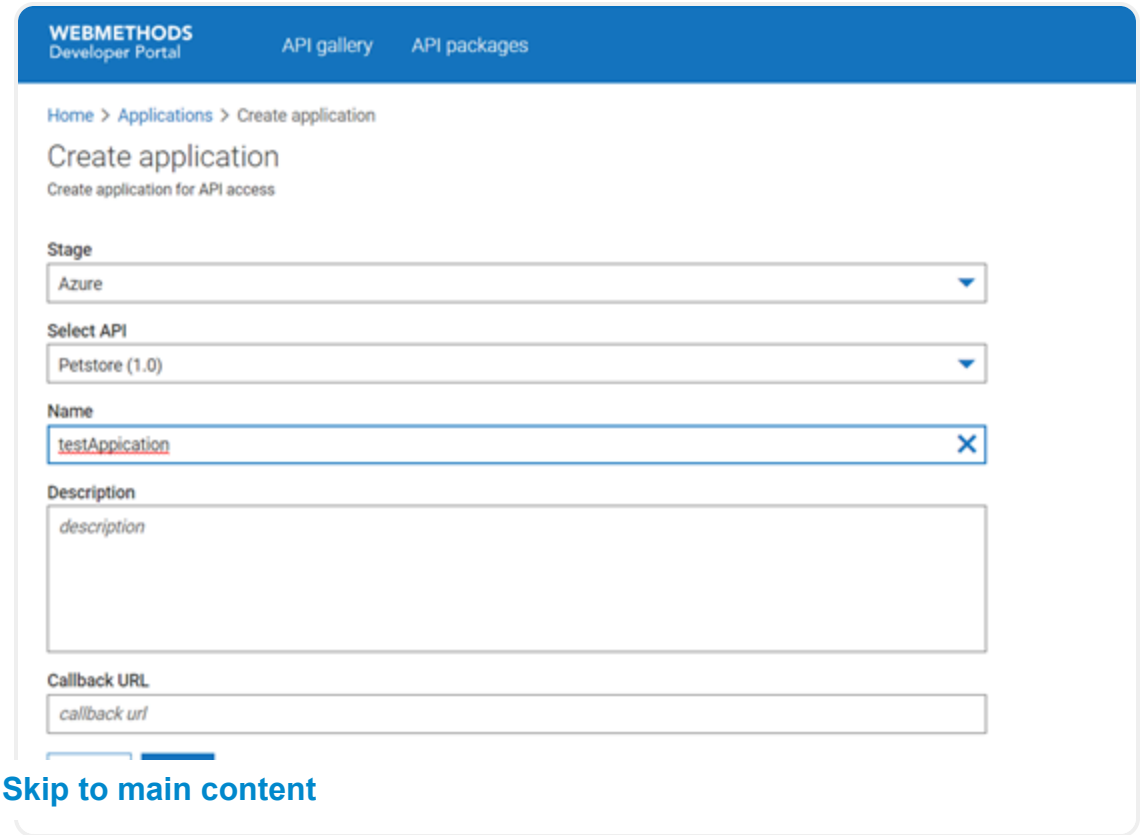
ag the newly imported API to the provider by clicking on Add below the APIs label.

5. Callbacks URL are configured as webhooks in developer portal. Click on Administration and webhooks tab. Enter the URL, select the type as Provider, security and event type and choose all the Gateway application event type.

Skip to main content

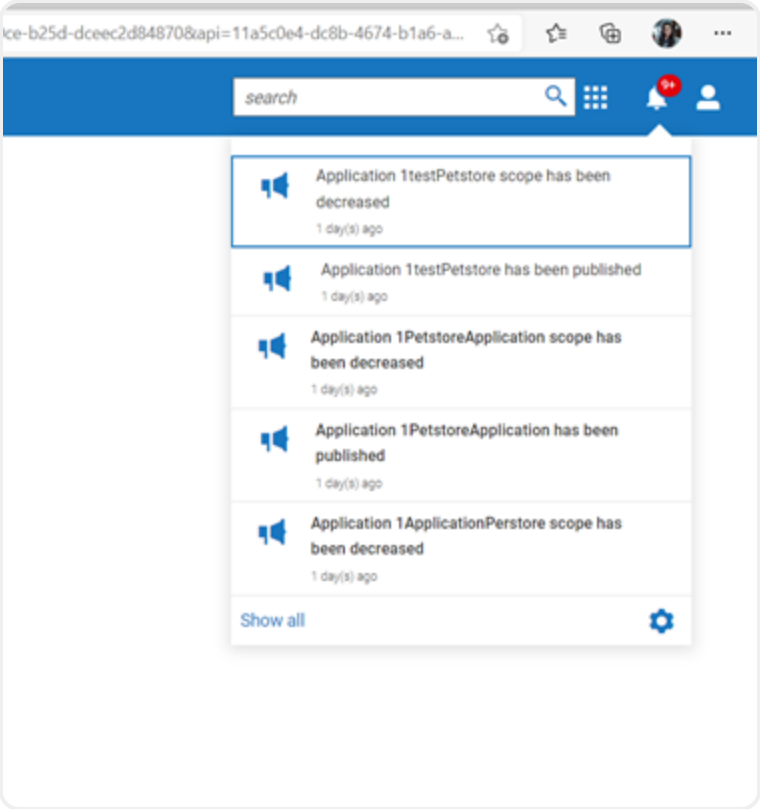


6. Go to the API and click on the Try API tab to test the API. Click on the create new application option. Enter an unique name for application, description and callback url is optional. If you have configured the callback url at the provider level you can ignore this field.



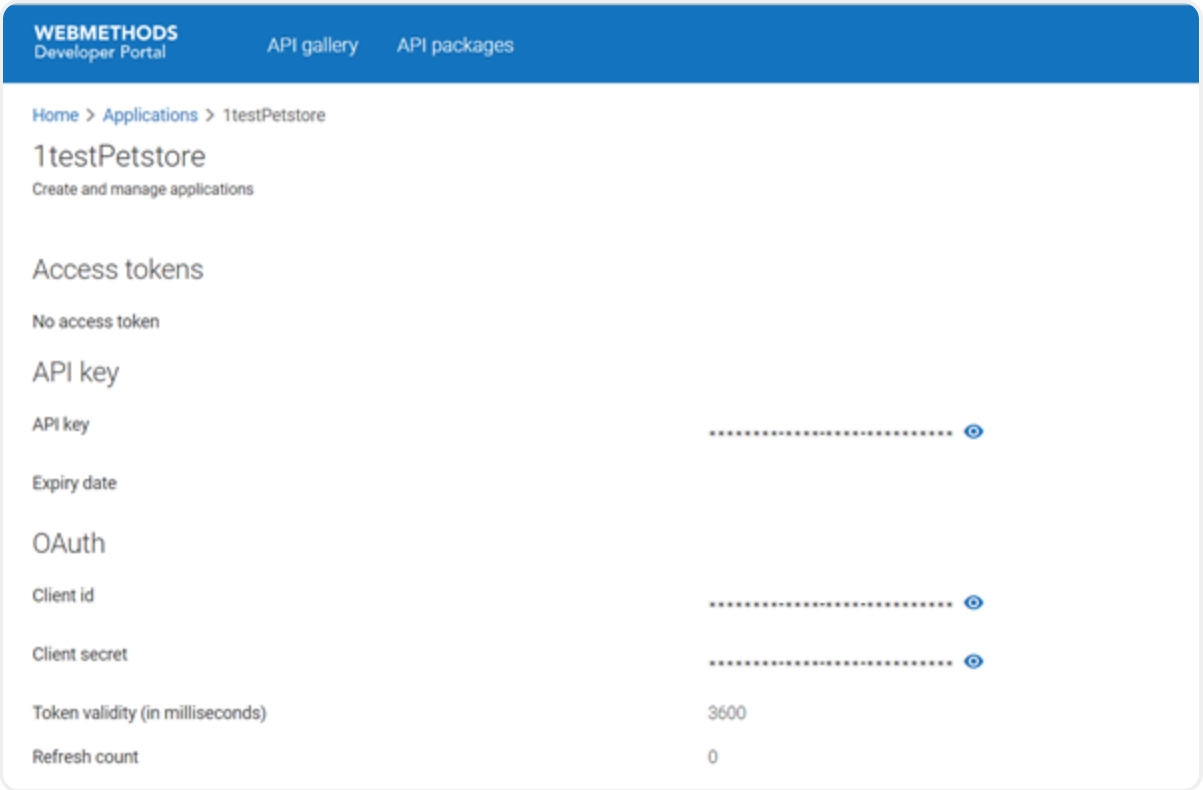
7. Once the application request is submitted the event will be published to the callback url. Integration logic should be written to create the subscription for this application at the Azure end. This integration logic is covered in the next section.

8. After the application is created the user will be notified in the notification section in developer portal.

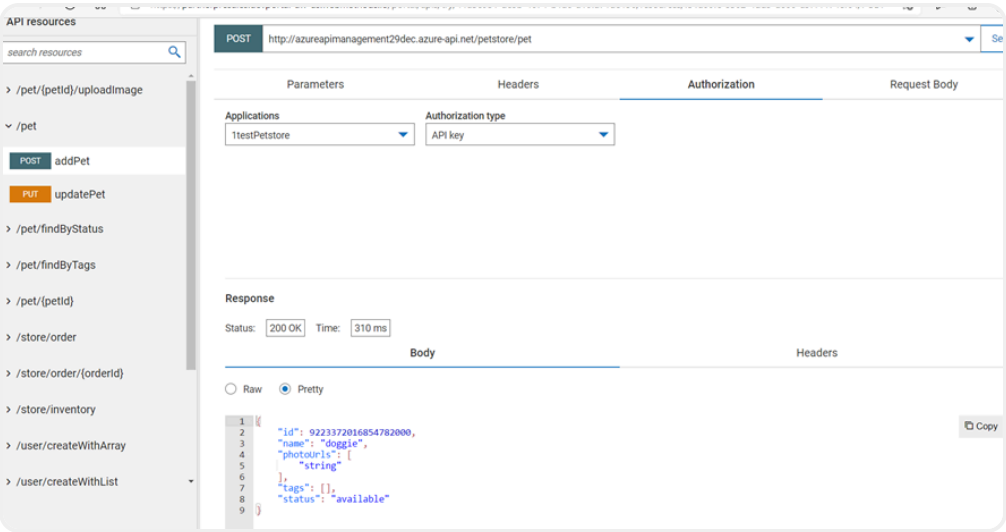


Skip to main content

9. You will be able to see the application details in the Manage applications section.



10. Now you can test the API using the application that was created recently.



ntegration Logic

elow mentioned integration logic needs to be built and any integration platform can be sed to build the logic. We have used **Webmethods.io**

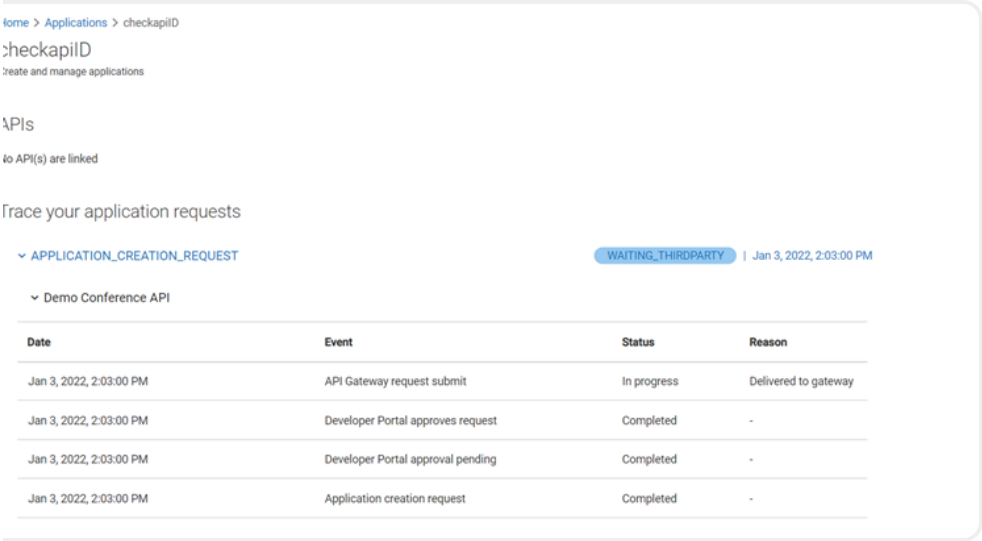
eq **Skip to main content** e:

1. Create a webhook and this url is the one which will be configured in developer portal. When an application is created in developer portal and an event will be published. Check for the event type and it will be REQUEST_ACCESS_TOKEN

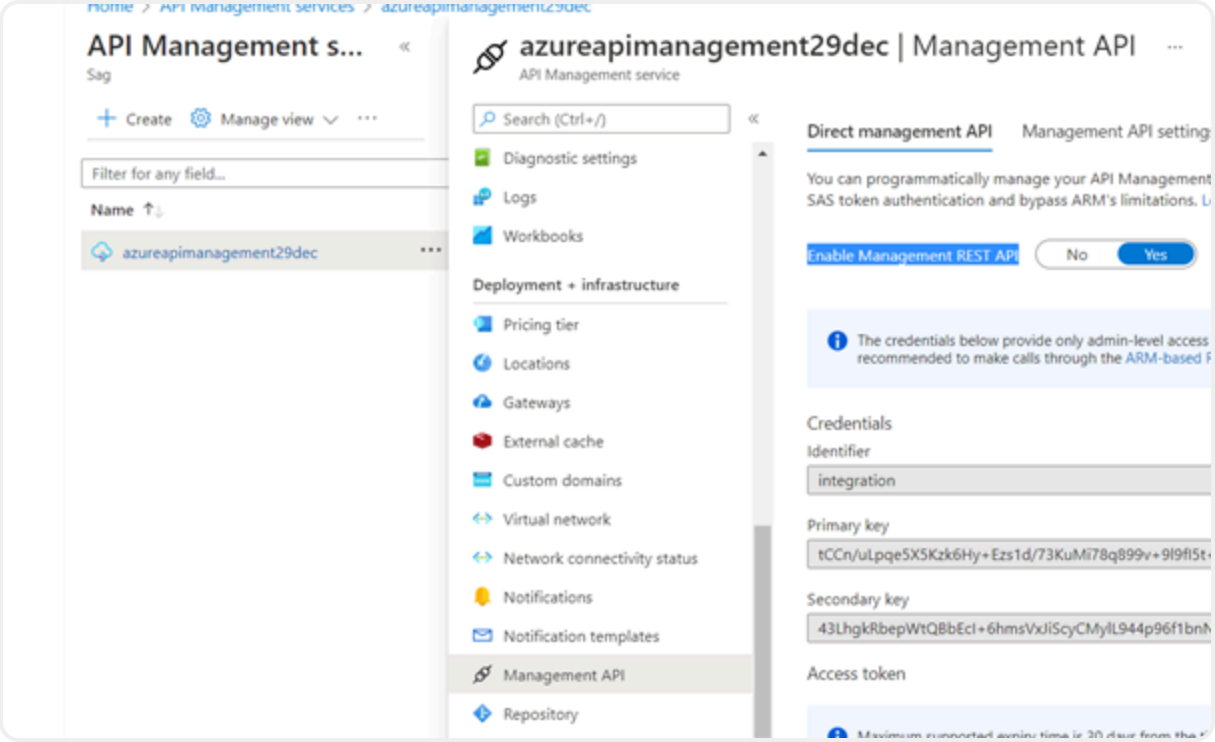

```
{
"executor":{
"id": "65ec5ddd-7f0c-34e0-91f9-815b684e0d41"
"email": "xxx"
"lastname": "xxx"
"firstname": "xxx"
"name": "xxxx"
}
"source":{
"id": "11a5c0e4-dc8b-4674-b1a6-a10fa71d8190"
"externalref": "335352c4-c285-4f6c-8bb2-e73ba6ed6d01"
}
"contextdata":{
"consumer_appName": "1testPetstore"
"consumer_appDesc": "test pet store"
"tenant": "xxxxx"
"redirect_uris":[
0: "https://xxxx/portal/rest/v1/oauth/callback"
]
"provider_id":NULL
"api.id": "11a5c0e4-dc8b-4674-b1a6-a10fa71d8190"
}
"type": "REQUEST_ACCESS_TOKEN"
"creationdate": "2022-01-05 08:56:43"
"eventid": "c352dd81-9fd2-4589-859a-2212615f97ee"
"status": "NEW"}
```


ne application creation event will be INPROGRESS state until its updated back using custom logic. In our case we will update after the subscription is completed in Azure ateway.

Skip to main content



2. To enable the REST API access in Azure API Management the Enable Management REST API should be enabled in Management API tab in Azure.



3. You need to get an authorization code from azure to make REST API calls. The below API can be used to get the bearer token.

Skip to main content

```
POST: /YOUR_AAD_TENANT/oauth2/token HTTP/1.1
Host: https://login.microsoftonline.com
Content-Type: application/x-www-form-urlencoded

grant_type= client_credentials
&client_id=YOUR_CLIENT_ID
&resource=  https://management.azure.com
&client_secret=YOUR_CLIENT_SECRET
```

The response of the API call will give you the access token. This token can be used to make further calls.

4. Next step is to create application in Azure which can be subscription to a API or subscription to a product(group of API's). We will create a subscription for an API using the below rest API.

```
PUT
https://management.azure.com/subscriptions/{subscriptionId}/resourceGroups/{resourceGroupName}/providers/Microsoft.ApiManagement/service/{serviceName}/subscriptions/{sid}?api-version=2021-08-01

Authorization Type:Bearer Token received from step 3.

Sample Request:
        {
          "properties": {
            "displayName":"Application ID given in developer Portal",
            "scope": "/
        subscriptions/{subscriptionId}/resourceGroups/{resourceGroupName}/providers
        /Microsoft.ApiManagement/service/{serviceName}/apis",
            "ownerId":
        subscriptions/{subscriptionId}/resourceGroups/{resourceGroupName}/providers
        /Microsoft.ApiManagement/service/{serviceName}/users/{userId}",
            "state": "active",
            "allowTracing": false
          }
}

Sample Response:
        {
        "id": "xxx"
        "type": "Microsoft.ApiManagement/service/subscriptions"
        "name": "testSubscription"
        "properties":{
        "ownerId": " xxx "
        "user":{
        "id": "xxx "
        "firstName": "Administrator"
        "lastName": ""
        "email": "avudaiselvi.90@gmail.com"
        "state": "active"
        "registrationDate": "2021-12-30T09:11:30.687Z"
        "note": NULL
        "groups":[]
        "identities":[]
        }
        "scope": " xxx"
        "displayName": " xxx "
        "state": "active"
        "createdDate": "2022-01-02T18:23:09.807Z"
        "startDate": "2022-01-02T00:00:00Z"
        "expirationDate":NULL
        "endDate":NULL
        "notificationDate":NULL
        "primaryKey": "xxx"
        "secondaryKey": " xxx"
        "stateComment":NULL
        "allowTracing":false
        }
}
```

Skip to main content

5. You primaryKey/secondaryKey will be the subscription key for the API that needs sent while testing the API. Incase if the API is OAuth enabled then the client Id and client secret along with the redirection URL needs to be updated in Developer Portal.

6. The event will be in progress state in developer portal so we will update the event first in developer portal using the eventId that's highlighted in step1 using the below rest API.

```
**Sample Request:**
**POST**: **https://xxxx/portal/abs/apirepository/v1/accesstokens/?eventId=even

Basic Authentication
{
    "name": "Application name",
    "owner": "Owner ID",
    "documentType": "APPLICATION",
    "providerRef": "Provider reference ID",
    "access": {
        "teams": [],
        "users": []
    },
    "apis": [
        "API ID can be retrieved from the event in step1"
    ],
    "credentials": [
        {
            "expiry": null,
            "apiKey": "primaryKey/secondaryKey from step 4",
            "type": "APIKey"
        },
        {
            "clientId": "clientId",
            "clientSecret": "clientSecret",
            "scopes": [],
            "tokenLifeTime": 3600,
            "tokenRefreshLimit": 0,
            "authorizationUris": [
                "https://login.microsoftonline.com/xxx/oauth2/authorize"
            ],
            "accessTokenUris": [
                "https://login.microsoftonline.com/xxx/oauth2/token"
            ],
            "redirectUris": [
                 "https://xxxx/portal/rest/v1/oauth/callback"
            ],
            "refreshTokenUris": [],
```

Skip to main content

```
        }
```

```
    ],
    "status": "LIVE",
    "slots": {
        "$stage": "lean portal"
    },
    "deleted": false,
    "app_type": "API"
}

Sample Response:
{
    "message": "Application created successfully",
    "accessTokenUuid": "a692fd03-af30-4120-8ea4-a9b86ea67e9f",
    "status": "SUCCESS"
}
```

7. Once the event is completed we have to update the application that's created in developer portal with the appropriate client credentials that's generated in Azure end. The event will have the applicationId(accessTokenUuid) in the response. You that id and with the same request as in step 6 update the application details using the below API.

```
**PUT: https://xxxxx/portal/rest/v1/applications/applicationId**

Basic Authentication

Sample Request:

{
    "name": "Application name",
    "owner": "Owner ID",
    "documentType": "APPLICATION",
    "providerRef": "Provider reference ID",
    "access": {
        "teams": [],
        "users": []
    },
    "apis": [
        "API ID can be retrieved from the event in step1"
    ],
    "credentials": [
        {
            "expiry": null,
            "apiKey": "primaryKey/secondaryKey from step 4",
            "type": "APIKey"
        },
        {
            "clientId": "clientId",
            "clientSecret": "clientSecret",
            "scopes": [],
```

Skip to main content

```
            "tokenLifeTime": 3600,
            "tokenRefreshLimit": 0,
            "authorizationUris": [
                "https://login.microsoftonline.com/xxx/oauth2/authorize"
            ],
            "accessTokenUris": [
                "https://login.microsoftonline.com/xxx/oauth2/token"
            ],
            "redirectUris": [
                 "https://xxxx.webmethods.io/portal/rest/v1/oauth/callback"
            ],
            "refreshTokenUris": [],
            "type": "OAuth2"
        }
    ],
    "status": "LIVE",
    "slots": {
        "$stage": "lean portal"
    },
    "deleted": false,
    "app_type": "API"
}
```


8. Step number 6 and 7 will be the same regardless of whatever backend API gateway provider is. The logic of creating the subscription at gateway end and retrieving the credentials will vary based on the API gateway provider.

Comments

Pascal Van Hove Jan 2022

Very insightful and useful. Thanks for sharing

---

anon41153502 Jan 2022

Awesome feature of Developer Portal showcasing the capabilities of Enterprise portal!

Arul Pragasam Rajendran Jan 2022

Very nice writeup!

Skip to main content

Meer Arfath Oct 2022

Very useful.

Related topics

Topic	Replies	Likes	Activity
Azure AD and OpenID Connect with webMethods API Gateway	4	1	Jun 2023
API access use cases on webMethods.io Developer Portal #newsletter	0	2	Apr 2022
Securing APIs using 3rd party OAuth 2 provider in API Gateway	2		Feb 2024
Debuging application requests raised from API Portal	0		Apr 2019
API Gateway topics on key features in API Gateway	0		Feb 2021

Software AG Tech Forums

Collaborate with your peers — developers, consultants, system integrators and business analysts from around the world. Join the Software AG Tech Community

Skip to main content

Join

Register

First Steps

Be a top contributor

Members

Explore

Search

Latest topics

Most popular

Feedback

Try for Free

Adabas & Natural

webMethods Integration

wM Service Designer

Software AG Cloud

Stay tuned

Subscribe to newsletter

Subscribe to notifications

Subscribe by product

Privacy Policy Terms of service About FAQs